

C++ Programming

A practical study reference with foundational concepts, examples, and common review questions.

1. Modern C++

C++ is a compiled, multi-paradigm language. Select an explicit project standard such as C++17 or C++20. Prefer standard-library facilities: `std::string` for text, `std::vector` for dynamic sequences, and algorithms for common operations.

A reference is an alias that must bind to an object; a pointer can be reseated and can represent no object. Use `const` references for large read-only inputs. Classes combine data and operations; constructors establish invariants and destructors release owned resources.

2. RAI and Ownership

RAII ties resource lifetime to object lifetime, so destructors release resources even during exception unwinding. Use `std::unique_ptr` for exclusive ownership; use `std::shared_ptr` only when shared lifetime is needed. Raw pointers and references are usually non-owning views.

The Rule of Zero favors composing types that already manage resources. If a type directly owns a resource, define copy and move behavior consistently. A moved-from object remains valid, but its value is usually unspecified.

3. Templates and Containers

Templates enable generic functions and types. `vector` suits contiguous dynamic sequences, `map` gives ordered key lookup, `unordered_map` gives hash-based lookup, and `set` stores unique values. Choose by required operations and complexity; iterator invalidation rules vary by container.

Algorithms such as `sort`, `find`, and `transform` operate on iterator ranges. Exceptions communicate failures that cannot be handled locally; throw by value and catch by `const` reference. A data race on unsynchronized shared memory is undefined behavior.

```
std::vector<int> scores{82, 95, 71};
std::sort(scores.begin(), scores.end());
auto high = std::find_if(scores.begin(), scores.end(),
    [](int score) { return score >= 90; });
```

4. Common Questions

RAII means resource acquisition is tied to initialization and deterministic destruction. A pointer may be null and reseated; a reference aliases an initialized object. Prefer smart pointers for ownership, compiler warnings and sanitizers for checks, and clear lifetime contracts in APIs.